

# **Design and Evaluation of a Distributed Hybrid System for Low-Latency Semantic and Structured Information Retrieval**



**UNIVERSITY  
OF LONDON**

Project proposal by Devang Chauhan  
Msc Computer Science

2481 Words

Excluding title page, table of content, references and appendix

# Table of Contents

|                                  |    |
|----------------------------------|----|
| Introduction.....                | 3  |
| Literature Review .....          | 4  |
| Presentation of the Problem..... | 7  |
| Proposed Solution .....          | 7  |
| Aims and Objectives.....         | 10 |
| Aims.....                        | 10 |
| Objectives.....                  | 10 |
| Development Plan .....           | 11 |
| References .....                 | 13 |

# Introduction

The advent of large language models (LLMs) have opened up numerous applications where a conversation can be had with an LLM using the knowledge encoded in the parameters of the model during pre-training. While fascinating, this has a limitation when it comes to their use within enterprises. Enterprise data is often gated and hence is not included in the pre-training data of the LLMs. This can lead to hallucinations where the LLM confidently responds to queries with false information where it does not have innate knowledge. Retrieval augmented generation (RAG), while based on techniques long known in the domains of search and recommendations systems, has been found to be a reliable way to ground the responses of large language model (LLM)s in relevant and recent information to reduce hallucinations [1].

RAG is extensively used to provide gated enterprise knowledge to LLMs to ground their responses. Depending on the shape of information the enterprise knowledge is often distributed across several knowledge stores such as vector stores, document stores, relational tables and graph databases. As this knowledge grows, the knowledge retrieval systems start taking the form of a distributed system where these knowledge stores needs to be either partitioned or replicated or both depending on whether consistency or availability is targeted [2].

In any such information retrieval system, it is important to maintain a balance between the speed and accuracy of the response served to the user to maintain a higher degree of user delight [3]. Therefore, the aim of this project is to evaluate how different orchestration and routing strategies affect latency, and retrieval quality in distributed hybrid retrieval systems. In this project, hybrid retrieval refers to combining semantic vector retrieval with structured relational querying within a unified orchestration.

This report begins by reviewing the work in this area. It then moves on to a clear presentation of the problem, followed by the aim and objectives, and a high level development plan.

# Literature Review

This project lies at the intersection of retrieval-augmented generation (RAG), natural language to SQL (NL2SQL), federated search, vector search, relational databases and distributed systems. Consequently, the literature review draws upon contributions from each of these domains to motivate the proposed architecture and identify the research gap.

RAG has received considerable attention from the research community. Xu et al. [4] propose a distributed RAG (DRAG) system using a peer-to-peer (P2P) network of knowledge stores and local LLMs. Their healthcare-focused approach introduces a novel algorithm that retrieves information locally before querying neighbouring peers. While the system is distributed, it relies solely on vector embeddings as grounding information for LLMs and therefore does not represent hybrid retrieval systems (HRS), where structured data must also be queried. Similarly, Guerraoui et al. [5] acknowledge the distributed nature of HRS and propose RAGROUTE, a neural network classifier that predicts which knowledge store should be queried. Although this reduces unnecessary communication, the routing model is probabilistic and requires retraining whenever the underlying knowledge stores are updated. Recognising that enterprise retrieval extends beyond embeddings, Yu et al. [6] propose TableRAG, which embeds relational table schemas alongside textual content so that retrieved schema embeddings can guide subsequent SQL queries. While this demonstrates the value of hybrid retrieval, the paper does not discuss how schema evolution or very large distributed tables are handled in practice.

As structured data becomes an increasingly important component of HRS, the quality of natural language to SQL (NL2SQL) translation has a direct impact on retrieval quality. Existing approaches can broadly be categorised as rule-based, neural and hybrid methods. Shalaan et al. [7] propose Guided SQL (GSQL), which combines rule-based processing, natural language processing and vector embeddings to improve SQL generation. SQLizer adopts a different approach by first constructing a query sketch before iteratively materialising it against the database schema using CoreNLP and Word2Vec [8]. More recently, Liu et al. [9] observe that although rule-based systems provide explainability, they struggle with adaptability and complex schemas, while neural approaches remain constrained by training data. Consequently, the field has shifted towards LLM-based methods and fine-tuning techniques that demonstrate stronger generalisation across complex database schemas [9]. Given this direction, implementing a custom NL2SQL engine would

contribute relatively little to the primary aim of this project, which is the evaluation of distributed query orchestration rather than query translation itself.

Once queries have been translated, the retrieval system must determine which resources should be searched. Federated search addresses this problem by providing a unified interface through which multiple independent collections can be searched and their results combined [10]. Garba et al. [10] identify CORI [11] and ReDDE [12] as two of the most influential resource selection techniques. Callan et al. [11] use probabilistic inference networks to estimate the likelihood that a collection contains relevant information, while CORI relies primarily on term frequency (TF) and inverse document frequency (IDF), limiting its ability to capture semantic similarity. Si et al. [12] improve resource selection by estimating collection sizes and emphasise that reducing communication between distributed collections becomes increasingly important as the number of knowledge stores grows. These studies establish that resource selection remains a fundamental challenge in distributed retrieval, although most existing approaches were developed for keyword-based search rather than modern semantic retrieval systems.

Semantic retrieval addresses this limitation by representing documents and queries as vector embeddings, allowing similarity to be measured using distance functions such as cosine similarity or Euclidean distance rather than exact keyword matching [2]. However, efficiently searching large vector collections presents another challenge. Flat indexes provide exact retrieval but exhibit linear search complexity, making them unsuitable for large datasets [2]. Modern vector databases therefore employ Approximate Nearest Neighbour (ANN) indexes such as IVF and HNSW, which significantly reduce retrieval latency while maintaining high retrieval quality [2,13]. More recent approaches, including DiskANN and SPANN, further improve scalability by storing large portions of the index on SSDs while maintaining lightweight routing structures in memory, enabling efficient retrieval over billion-scale datasets [14,15]. These approaches substantially improve vector search performance but focus on optimising retrieval within individual vector stores rather than orchestration across heterogeneous distributed knowledge stores.

While semantic retrieval addresses unstructured information, enterprise knowledge also resides within relational databases. Relational systems typically employ B-tree indexes for range queries, hash indexes for equality comparisons and GIN indexes for full-text and JSON search [2,16]. Kleppmann and Riccomini [2] note that although indexes improve retrieval performance, they increase storage requirements and slow write operations because they must be maintained as the underlying data changes.

Consequently, index selection represents a trade-off between query performance and maintenance overhead.

As enterprise knowledge continues to grow, a single machine may no longer be sufficient to store or process all data. Distributed database systems therefore introduce concerns such as sharding and replication. The degree of fragmentation directly influences query performance [17], while the choice of sharding strategy depends heavily on workload characteristics. Key-range partitioning is effective when queries exhibit locality, whereas hash-based partitioning generally provides a more balanced distribution of reads and writes [2]. Similar principles apply to vector databases, where semantically similar vectors may be clustered together to reduce the search space during retrieval [18]. Distributed systems also introduce opportunities for parallel query execution; however, Özsu and Valduriez [19] note that reducing response time through parallelism may increase overall execution time because of additional communication overhead. Consequently, computation should occur as close to the data as possible whenever feasible [19].

These considerations ultimately influence query planning within distributed retrieval systems. Özsu and Valduriez [20] argue that query optimisation should minimise communication costs by reducing the amount of data transferred between nodes. They recommend assigning execution costs to alternative query plans and favour strategies that perform selection as early as possible, thereby reducing intermediate results and avoiding expensive join operations wherever possible [20]. Such observations reinforce the importance of orchestration strategies that intelligently coordinate retrieval across multiple heterogeneous knowledge stores.

Collectively, the literature demonstrates significant progress in semantic retrieval, distributed RAG systems, vector indexing and hybrid retrieval. Existing work has largely focused on improving individual components of the retrieval pipeline, including vector search, NL2SQL translation and resource selection. Comparatively little research, however, evaluates how deterministic orchestration strategies influence latency and retrieval effectiveness in distributed hybrid retrieval systems that combine semantic vector search with structured relational querying. This project addresses that gap by designing and experimentally evaluating alternative routing and orchestration strategies under different sharding and replication configurations using established information retrieval and distributed systems performance metrics.

# Presentation of the Problem

Enterprise retrieval systems increasingly combine semantic vector search with structured relational querying over distributed knowledge stores. While previous research has produced significant advances in distributed RAG architectures, vector indexing, resource selection and NL2SQL, these contributions largely optimise individual components of the retrieval pipeline in isolation.

In enterprise environments, deterministic orchestration is often preferred because retrieval decisions are predictable and auditable, allowing system designers to understand why particular data sources were queried for a given request. However, the performance implications of adopting deterministic orchestration strategies in distributed hybrid retrieval systems remain insufficiently understood.

Little work evaluates how different deterministic orchestration strategies influence end-to-end retrieval latency and retrieval quality when semantic and structured retrieval must be coordinated across distributed knowledge stores. Consequently, practitioners have limited evidence regarding which orchestration strategies provide the best trade-off between latency and retrieval quality under different sharding and replication configurations.

In any information retrieval system, achieving user delight through a careful balance between speed and relevance of response is perhaps the most important concern [3]. This project addresses that concern through the design and experimental evaluation of a distributed hybrid retrieval system.

## Proposed Solution

Distributed hybrid retrieval systems (HRS) receive natural language queries that may require information from multiple heterogeneous knowledge stores. The role of the orchestration layer is to determine which knowledge stores should be queried, in what order they should be queried, and how the retrieved results should be combined before returning a response. In distributed environments, these decisions directly influence communication overhead, response latency and retrieval quality.

HRS often encounters complex queries. An example of a complex query would be:

"Give us three movie options directed by an academy award-winning director that will motivate us to survive through tough times in life."

Such a query needs to be decomposed into the following sub-queries:

Query 1: Find movies directed by Academy Award-winning directors.

Query 2: Among those movies, identify ones whose plot summaries describe inspirational stories.

In enterprise applications, LLMs are increasingly used to generate such sub-queries from a complex user query and to synthesise a final response from the retrieved information [21]. These components, however, are not the focus of this project and are therefore kept outside the scope of the evaluation [Figure 1].

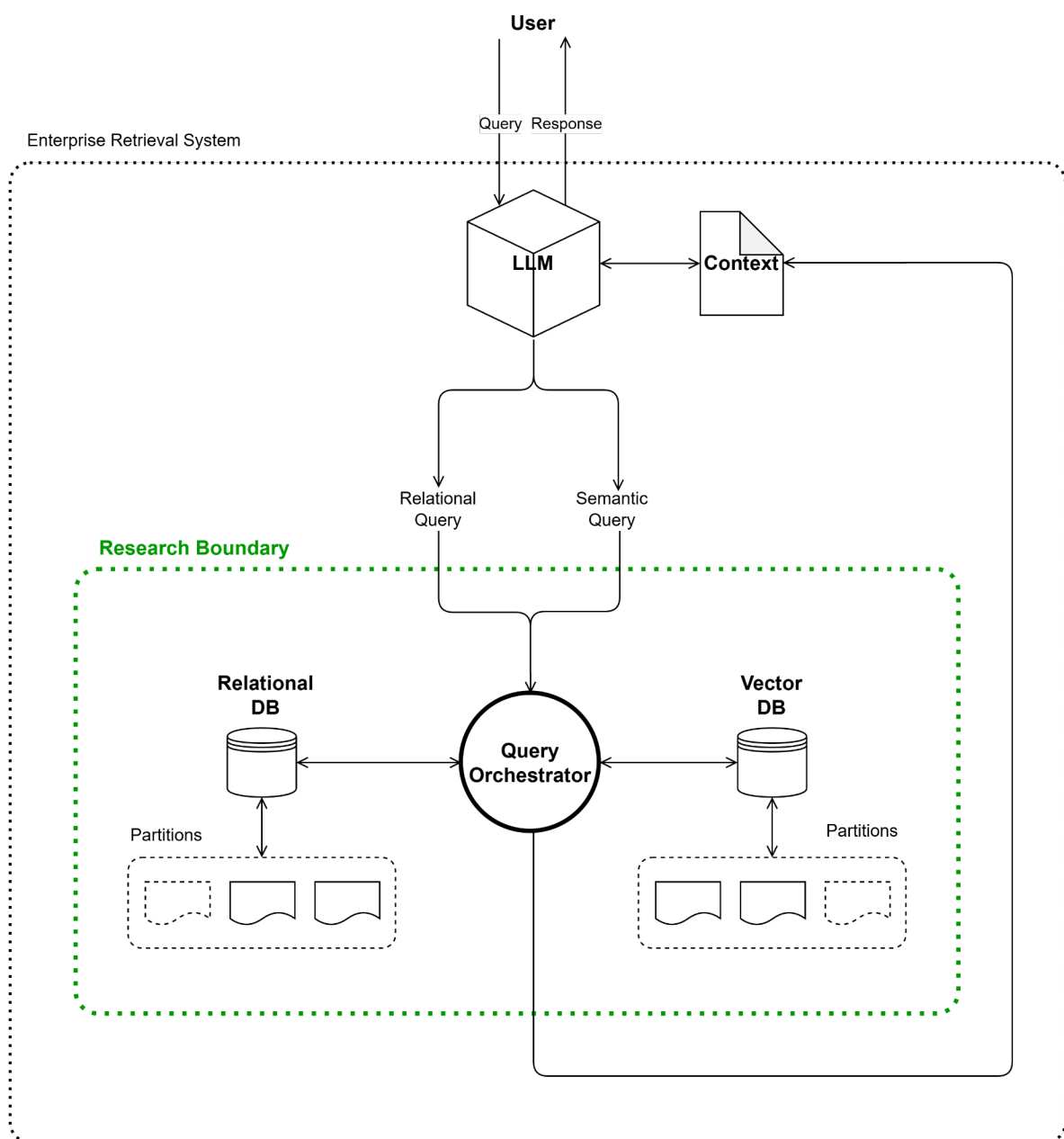


Figure - 1 Research Boundary

Instead, this project focuses on the deterministic orchestration layer responsible for coordinating retrieval across distributed knowledge stores. A deterministic rule-based approach is adopted because routing decisions are transparent, reproducible and easier to analyse experimentally than learned routing models. This also enables controlled comparison of different orchestration strategies under identical distributed environments.

The project proposes to design and implement a query orchestration framework as a REST API written in Rust. Since Rust is a statically typed, compiled language without a garbage collector, it is well suited for implementing latency-sensitive distributed services.

The orchestrator will potentially include:

- Rule-based query parsing and routing logic
- Evaluation of multiple retrieval strategies, including:
  - Broadcast querying
  - Selective shard querying
  - Filter-then-search
  - Search-then-filter
- Retrieval coordination
- Distributed shard coordination
- Replication and node-failure experimentation

Each routing strategy will be implemented within the same orchestration framework, allowing their latency and retrieval effectiveness to be compared under different sharding and replication configurations.

Only a limited number of REST endpoints will be implemented, such as query execution and node health monitoring, since the contribution of the project lies in the orchestration layer rather than in developing a complete CRUD application.

# Aims and Objectives

## Aims

To design and experimentally evaluate a distributed hybrid query orchestration framework, and determine how different deterministic routing strategies influence retrieval latency and retrieval quality under distributed conditions.

## Objectives

1. Design and implement a distributed query orchestration framework as a REST API capable of coordinating semantic vector retrieval and structured relational querying.
2. Implement and compare multiple deterministic routing strategies, including broadcast querying, selective shard querying, filter-then-search and search-then-filter.
3. Evaluate the impact of different sharding and replication configurations on the latency and retrieval effectiveness of the proposed orchestration framework.
4. Measure system performance using established distributed systems and information retrieval metrics, including:
  - Mean, P95 and P99 latency
  - Precision
  - Recall
  - F-measure
  - Normalised Discounted Cumulative Gain (NDCG)
  - Known-item retrieval accuracy
5. Analyse the trade-offs between routing strategies and identify the conditions under which each strategy provides the best balance between retrieval latency and retrieval quality.

# Development Plan

The project work is planned to progress in a sequential manner with a week being the smallest unit [Figure-2].

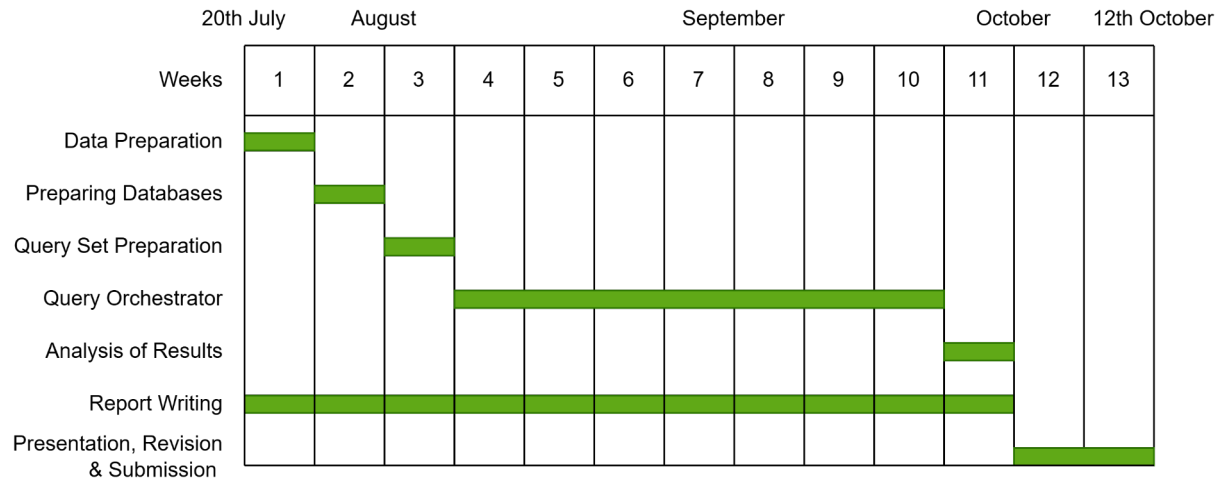


Figure - 2 Timeline

Publicly available movie datasets, including IMDb [22], MovieLens [23], TMDb [24,26] and Wikipedia Movie Plots [25], will be evaluated to construct the relational and semantic knowledge stores. Structured metadata will be stored in PostgreSQL [27], while plot embeddings will be stored in Qdrant [28]. Docker Containerisation will be used to create local instances of the databases, with cloud based deployments reserved for a future extension.

As per the scope [Figure-3] of this project, a query  $Q$  can be of following three types, and a response is  $R$ .

- Q1: relational query
- Q2: semantic query
- Q3: complex query. A tuple of relational query and semantic query.

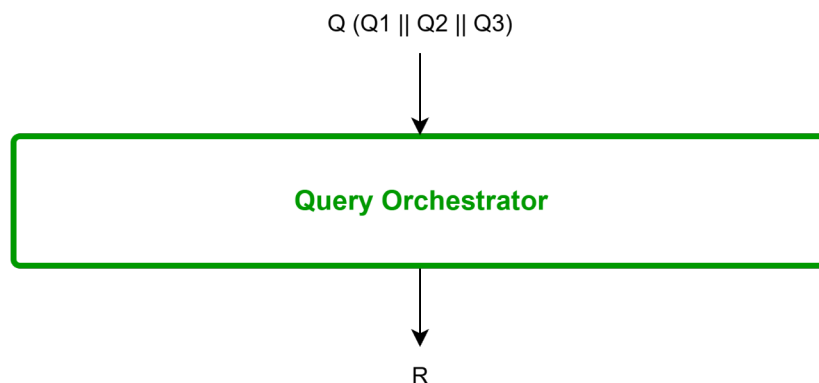


Figure - 3 Scope of Query Orchestrator

Two independent benchmark query sets with corresponding ground-truth responses will be created. Each set will contain all the three types of queries in equal amount. The first set will be used for developing the orchestrator, leaving the other one for final evaluation.

Based on the query set, an appropriate partitioning strategy will be selected before constructing the databases. While populating the relational knowledge store will be relatively straightforward, a single pretrained embedding model (OpenAI's text-embedding-3-small [29]) will be used throughout all experiments to generate embeddings for semantic queries and movie plot summaries, isolating the effects of the orchestration strategies.

Once gold standard queries and the databases are in place the query orchestrator as a REST API will be developed using the first set of queries. At the conclusion of the development, the orchestrator will be evaluated against the remaining set of the query set.

Each routing strategy will be executed against the same benchmark workload to ensure that any observed differences in latency or retrieval effectiveness arise from the orchestration strategy rather than variations in the test data.

Report writing will continue throughout to capture the findings along the way, leaving the last couple of weeks for the presentation and revision.

# References

- [1] C. Huyen, *AI Engineering: Building Applications with Foundation Models*. Sebastopol, CA: O'Reilly Media, 2025.
- [2] M. Kleppmann and C. Riccomini, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 2nd edition. Santa Rosa, CA: O'Reilly Media, 2026.
- [3] C. D. Manning, P. Raghavan, and H. Schütze, "Evaluation in information retrieval," in *Introduction to Information Retrieval*, Cambridge University Press, pp. 151–175. [Online]. Available: <https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-in-information-retrieval-1.html>
- [4] C. Xu, L. Gao, Y. Miao, and X. Zheng, "Distributed Retrieval-Augmented Generation." Accessed: Jun. 01, 2026. [Online]. Available: <https://arxiv.org/html/2505.00443v1>
- [5] R. Guerraoui, A.-M. Kermarrec, D. Petrescu, R. Pires, M. Randl, and M. de Vos, "Efficient Federated Search for Retrieval-Augmented Generation," in *Proceedings of the 5th Workshop on Machine Learning and Systems*, in EuroMLSys '25. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 74–81. doi: 10.1145/3721146.3721942.
- [6] X. Yu, P. Jian, and C. Chen, "TableRAG: A Retrieval Augmented Generation Framework for Heterogeneous Document Reasoning," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds., Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 14063–14082. doi: 10.18653/v1/2025.emnlp-main.710.
- [7] H. S. Shalaan, T. H. A. Soliman, and A. M. AbdelAziz, "G-SQL: A Schema-Aware and Rule-Guided Approach for Robust Natural Language to SQL Translation," *IEEE Access*, vol. 13, pp. 158520–158534, 2025, doi: 10.1109/ACCESS.2025.3607879.
- [8] N. Yaghmazadeh, Y. Wang, I. Dillig, and T. Dillig, "SQLizer: query synthesis from natural language," *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, p. 63:1-63:26, Oct. 2017, doi: 10.1145/3133887.
- [9] X. Liu et al., "A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and

Where Are We Going?," IEEE Transactions on Knowledge and Data Engineering, vol. 37, no. 10, pp. 5735–5754, Oct. 2025, doi: 10.1109/TKDE.2025.3592032.

[10] A. Garba, S. Wu, and S. Khalid, "Federated search techniques: an overview of the trends and state of the art," Knowl Inf Syst, vol. 65, no. 12, pp. 5065–5095, Dec. 2023, doi: 10.1007/s10115-023-01922-6.

[11] J. P. Callan, Z. Lu, and W. B. Croft, "Searching distributed collections with inference networks," in Proceedings of the 18th annual international ACM SIGIR conference on Research and development in information retrieval, in SIGIR '95. New York, NY, USA: Association for Computing Machinery, Jul. 1995, pp. 21–28. doi: 10.1145/215206.215328.

[12] L. Si and J. Callan, "Relevant document distribution estimation method for resource selection," in Proceedings of the 26th annual international ACM SIGIR conference on Research and development in information retrieval, in SIGIR '03. New York, NY, USA: Association for Computing Machinery, Jul. 2003, pp. 298–305. doi: 10.1145/860435.860490.

[13] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[14] Q. Chen et al., "SPANN: highly-efficient billion-scale approximate nearest neighbor search," in Proceedings of the 35th International Conference on Neural Information Processing Systems, in NIPS '21. Red Hook, NY, USA: Curran Associates Inc., Dec. 2021, pp. 5199–5212.

[15] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node," in Advances in Neural Information Processing Systems, Curran Associates, Inc., 2019. Accessed: Jun. 25, 2026. [Online]. Available: [https://papers.nips.cc/paper\\_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html](https://papers.nips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html)

[16] "PostgreSQL 18.4 Documentation," PostgreSQL Documentation. Accessed: Jun. 11, 2026. [Online]. Available: <https://www.postgresql.org/docs/18/index.html>

[17] M. T. Özsu and P. Valduriez, "Distributed and Parallel Database Design," in Principles of Distributed Database Systems, M. T. Özsu and P. Valduriez, Eds.,

Cham: Springer International Publishing, 2020, pp. 33–89. doi: 10.1007/978-3-030-26253-2\_2.

[18] J. J. Pan, J. Wang, and G. Li, “Survey of vector database management systems,” *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, Jul. 2024, doi: 10.1007/s00778-024-00864-x.

[19] M. T. Özsu and P. Valduriez, “Parallel Database Systems,” in *Principles of Distributed Database Systems*, M. T. Özsu and P. Valduriez, Eds., Cham: Springer International Publishing, 2020, pp. 349–394. doi: 10.1007/978-3-030-26253-2\_8.

[20] M. T. Özsu and P. Valduriez, “Distributed Query Processing,” in *Principles of Distributed Database Systems*, M. T. Özsu and P. Valduriez, Eds., Cham: Springer International Publishing, 2020, pp. 129–182. doi: 10.1007/978-3-030-26253-2\_4.

[21] haileytap, “Agentic Retrieval Overview - Azure AI Search.” Accessed: Jun. 11, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/search/agentic-retrieval-overview>

[22] “IMDb Non-Commercial Datasets,” IMDb Developer. Accessed: Jun. 12, 2026. [Online]. Available: <https://developer.imdb.com/non-commercial-datasets/>

[23] “MovieLens.” Accessed: Jun. 12, 2026. [Online]. Available: <https://movielens.org/>

[24] “TMDB API,” The Movie Database (TMDB). Accessed: Jul. 12, 2026. [Online]. Available: <https://developer.themoviedb.org/docs/getting-started>

[25] “Wikipedia Movie Plots.” Accessed: Jun. 12, 2026. [Online]. Available: <https://www.kaggle.com/datasets/jrobischon/wikipedia-movie-plots>

[26] “Full TMDB Movies Dataset 2024 (1M Movies).” Accessed: Jul. 12, 2026. [Online]. Available: <https://www.kaggle.com/datasets/asaniczka/tmdb-movies-dataset-2023-930k-movies>

[27] P. G. D. Group, “PostgreSQL,” PostgreSQL. Accessed: Jul. 12, 2026. [Online]. Available: <https://www.postgresql.org/>

[28] “Qdrant - Vector Search Engine.” Accessed: Jul. 12, 2026. [Online]. Available: <https://qdrant.tech/>

[29] “text-embedding-3-small Model | OpenAI API.” Accessed: Jul. 12, 2026. [Online]. Available: <https://developers.openai.com/api/docs/models/text-embedding-3-small>